

Java 7 — Try-with-Resources

Java Version: 7

Year: 2011

Feature: Try-with-Resources

UniqueClassName: TryWithResources

1. Definition

Try-with-resources is a Java 7 language feature that automatically closes resources after they are used.

A **resource** is an object that implements the `java.lang.AutoCloseable` interface, such as:

- `FileInputStream`
- `FileOutputStream`
- `BufferedReader`
- `BufferedWriter`
- `Scanner`
- `Connection`
- `Statement`
- `ResultSet`

Before Java 7, resources generally had to be closed manually inside a `finally` block.

Basic syntax

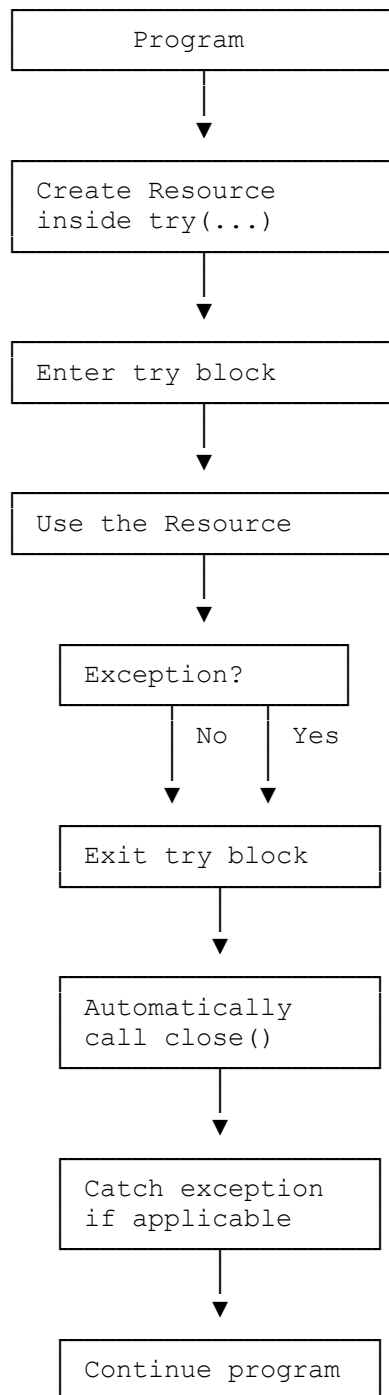
```
try (ResourceType resource = new ResourceType()) {  
    // Use resource  
}
```

When execution leaves the `try` block, Java automatically invokes:

```
resource.close();
```

even when an exception occurs.

2. Execution Flowchart



The important sequence is:

Create resource → use resource → leave try → automatically call close() → continue/catch exception

3. Java Code Example

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class TryWithResources {

    public static void main(String[] args) {

        try (BufferedReader reader =
            new BufferedReader(new FileReader("data.txt"))) {

            String line;

            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

    }
}
```

How it works

```
try (BufferedReader reader =
    new BufferedReader(new FileReader("data.txt"))) {
```

The `BufferedReader` is declared inside the parentheses after `try`.

Java automatically closes it when the `try` block finishes.

Conceptually, Java performs something similar to:

```
reader.close();
```

without requiring the programmer to explicitly write it.

4. Multiple Resources

Java 7 also allows multiple resources in the same `try` statement.

```
import java.io.*;

public class TryWithResources {
```

```

public static void main(String[] args) {

    try (
        FileReader fileReader = new FileReader("input.txt");
        BufferedReader reader = new BufferedReader(fileReader)
    ) {

        String line;

        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }

    } catch (IOException e) {
        e.printStackTrace();
    }

}

```

The resources are closed in **reverse order of creation**.

Creation order:

```

FileReader
↓
BufferedReader

```

Closing order:

```

BufferedReader
↓
FileReader

```

This is important because the outer/wrapper resource should generally be closed before the underlying resource.

5. Try-with-Resources vs Traditional try-finally

Feature	Traditional try-finally	Try-with-resources
Introduced	Before Java 7	Java 7
Resource closing	Manual	Automatic
Requires finally	Usually	No
Code size	Larger	Smaller

Exception handling	More complicated	Improved
AutoCloseable	Not required	Required
Multiple resources	More verbose	Easy
Suppressed exceptions	Difficult to handle manually	Automatically supported
Recommended	Legacy code	Modern approach

Traditional approach

```
BufferedReader reader = null;

try {
    reader = new BufferedReader(
        new FileReader("data.txt")
    );

    System.out.println(reader.readLine());
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Java 7 approach

```
try (BufferedReader reader =
    new BufferedReader(new FileReader("data.txt"))) {

    System.out.println(reader.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

The second version is significantly cleaner and safer.

6. AutoCloseable

Try-with-resources works with objects implementing:

`java.lang.AutoCloseable`

Example:

```
class MyResource implements AutoCloseable {  
    public void use() {  
        System.out.println("Using resource");  
    }  
  
    @Override  
    public void close() {  
        System.out.println("Resource closed");  
    }  
}
```

Use it:

```
public class TryWithResources {  
    public static void main(String[] args) {  
        try (MyResource resource = new MyResource()) {  
            resource.use();  
        }  
    }  
}
```

Output

Using resource
Resource closed

The programmer did not explicitly call:

`resource.close();`

Java did it automatically when the try block ended.

7. Important Execution Rule

Suppose:

```
try (MyResource resource = new MyResource()) {  
    System.out.println("Inside try");  
}
```

The logical execution is:

1. Create MyResource
↓
2. Store resource reference
↓
3. Enter try block
↓
4. Execute statements
↓
5. Leave try block
↓
6. Automatically invoke resource.close()
↓
7. Continue execution

If an exception occurs:

```
Create Resource  
↓  
Enter try  
↓  
Execute code  
↓  
Exception occurs  
↓  
Automatically close Resource  
↓  
Catch exception  
↓  
Continue / terminate
```

Therefore, resource cleanup happens before control is transferred to the appropriate catch/finally processing.

8. Suppressed Exceptions

One of the important advantages of try-with-resources is **suppressed exceptions**.

Consider:

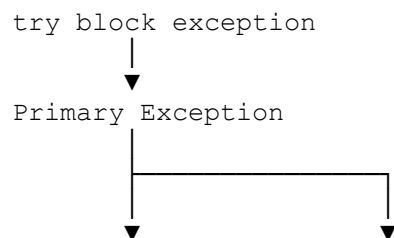
```
class MyResource implements AutoCloseable {  
    @Override  
    public void close() {  
        throw new RuntimeException("Exception from close()");  
    }  
}
```

And:

```
public class TryWithResources {  
    public static void main(String[] args) {  
        try (MyResource resource = new MyResource()) {  
            throw new RuntimeException("Exception from try");  
        } catch (RuntimeException e) {  
            System.out.println("Main exception: "  
                               + e.getMessage());  
            for (Throwable suppressed : e.getSuppressed()) {  
                System.out.println(  
                    "Suppressed exception: "  
                    + suppressed.getMessage()  
                );  
            }  
        }  
    }  
}
```

The exception from the try body is considered the **primary exception**, while the exception generated during close() can become a **suppressed exception**.

Conceptually:



close() executes close() exception
 ↓
 Suppressed Exception

You can retrieve suppressed exceptions using:

```
e.getSuppressed();
```

9. Key Points to Remember

Point	Explanation
Introduced	Java 7
Purpose	Automatic resource management
Syntax	try (resource) { }
Main interface	AutoCloseable
Automatic method	close()
Manual finally	Generally unnecessary
Multiple resources	Supported
Closing order	Reverse order of declaration
Exception during try	Resource is still closed
Exception during close()	Can become suppressed
Main benefit	Safer and cleaner resource management

One-line definition

Try-with-resources, introduced in Java 7, automatically closes AutoCloseable resources when the try block finishes, even when an exception occurs.

Java 7 — Multi-catch

Java Version: 7

Year: 2011

Feature: Multi-catch

UniqueClassName: MultiCatch

1. Definition

Multi-catch is a Java 7 feature that allows **multiple exception types to be handled by a single catch block**.

Before Java 7, if different exceptions required the same handling logic, separate catch blocks were normally needed.

Syntax

```
try {  
    // code that may throw exceptions  
}  
catch (ExceptionType1 | ExceptionType2 | ExceptionType3 e) {  
    // common exception handling  
}
```

The | operator separates the exception types.

2. Traditional Approach Before Java 7

Without multi-catch:

```
try {  
    // risky code  
}  
catch (ArithmeticException e) {  
    System.out.println("Error: " + e.getMessage());  
}  
catch (NumberFormatException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

Both exceptions have identical handling logic, resulting in duplicated code.

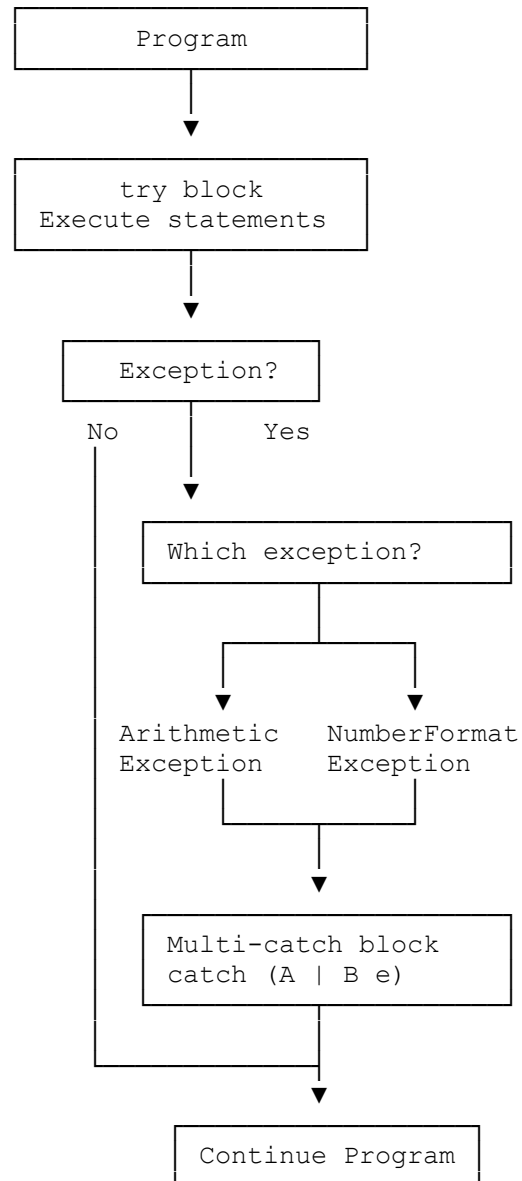
With Java 7:

```
try {  
    // risky code  
}
```

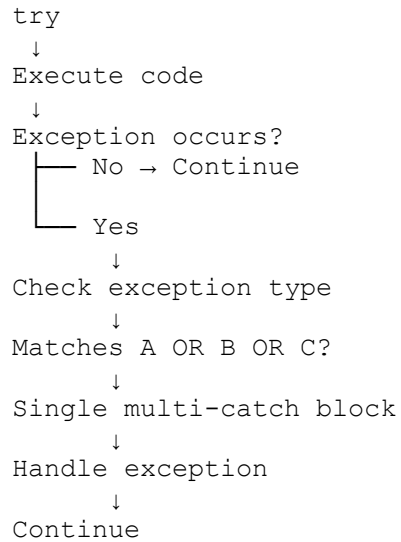
```
catch (ArithmeticException | NumberFormatException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

The common handling logic can be written only once.

3. Multi-catch Flowchart



Simplified execution sequence



4. Basic Java Code Example

```
public class MultiCatch {

    public static void main(String[] args) {

        try {

            int number = Integer.parseInt("ABC");
            int result = 100 / number;

            System.out.println(result);

        } catch (ArithmeticException | NumberFormatException e) {

            System.out.println("Exception occurred: "
                               + e.getMessage());

        }

    }

}
```

Output

Exception occurred: For input string: "ABC"

Here:

```
catch (ArithmeticException | NumberFormatException e)
```

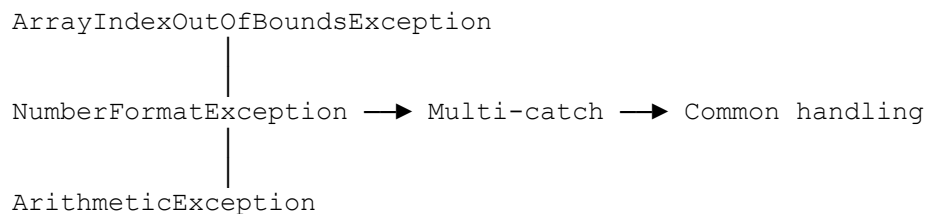
means:

If the exception is `ArithmeticException` **OR** `NumberFormatException`, execute this same catch block.

5. Example with Different Operations

```
public class MultiCatch {  
    public static void main(String[] args) {  
        try {  
            String value = args[0];  
            int number = Integer.parseInt(value);  
            int result = 100 / number;  
            System.out.println("Result = " + result);  
        } catch (ArrayIndexOutOfBoundsException  
                | NumberFormatException  
                | ArithmeticException e) {  
            System.out.println(  
                "Exception Type: "  
                + e.getClass().getSimpleName()  
            );  
            System.out.println(  
                "Message: "  
                + e.getMessage()  
            );  
        }  
    }  
}
```

Three different exceptions are handled by one catch block:



6. Important Rule: Exceptions Cannot Have a Parent-Child Relationship

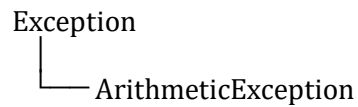
You **cannot** write:

```
catch (Exception | ArithmeticException e) {  
}
```

because `ArithmeticException` is already a subclass of `Exception`.

The compiler rejects this because the more specific exception is redundant.

Conceptually:



Therefore, use either:

```
catch (Exception e) {  
}
```

or:

```
catch (ArithmeticException e) {  
}
```

but not:

```
catch (Exception | ArithmeticException e) {  
}
```

7. Multi-catch Exception Variable

In:

```
catch (ArithmeticException | NumberFormatException e) {  
    System.out.println(e.getMessage());  
}
```

the variable `e` represents an exception whose type is one of the alternatives.

You can call methods common to the exception hierarchy, such as:

```
e.getMessage();  
e.getCause();
```

```
e.printStackTrace();  
e.toString();
```

For example:

```
catch (ArithmeticException | NumberFormatException e) {  
  
    System.out.println(e.getClass());  
    System.out.println(e.getMessage());  
}
```

8. Multi-catch Variable Is Effectively Final

A multi-catch exception parameter cannot be reassigned.

This is invalid:

```
catch (ArithmeticException | NumberFormatException e) {  
  
    e = new ArithmeticException();  
}
```

The compiler does not allow reassignment of the multi-catch parameter.

However, you can use the variable normally:

```
catch (ArithmeticException | NumberFormatException e) {  
  
    System.out.println(e.getMessage());  
}
```

9. Multi-catch vs Multiple catch Blocks

Feature	Multiple catch blocks	Multi-catch
Introduced	Before Java 7	Java 7
Number of handlers	Multiple	One
Syntax	Separate catch blocks	Exception1 Exception2
Common code	Can be duplicated	Written once
Different handling	Excellent	Not suitable if handling differs

Code duplication	Possible	Reduced
Readability	Can become verbose	Cleaner for common handling
Exception variable	Separate variable	One variable
Parent + child types	Allowed in separate ordered catches	Not allowed in same multi-catch

10. When to Use Multi-catch

Use multi-catch when **different exception types require the same handling logic**.

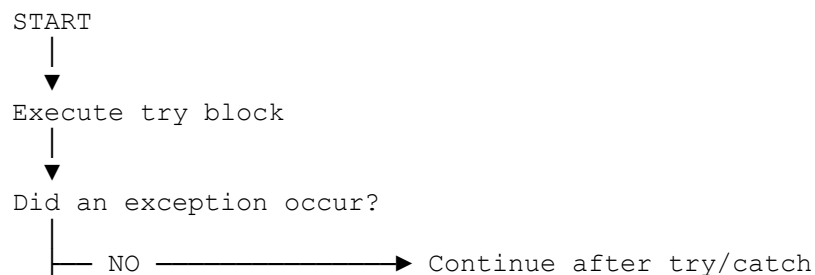
For example:

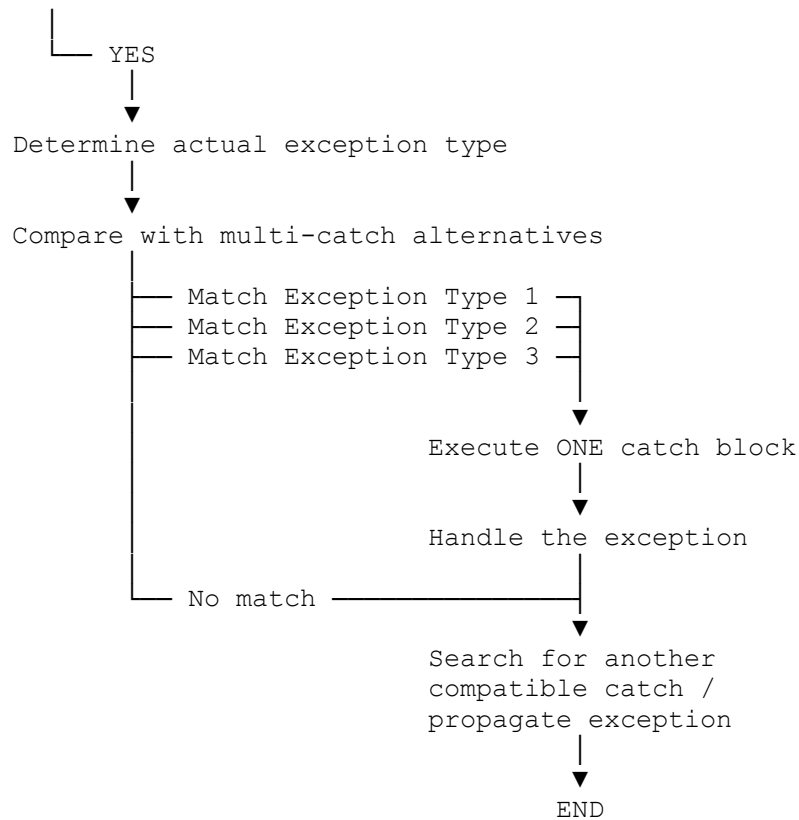
```
try {
    // operation
}
catch (IOException | SQLException e) {
    System.out.println("Unable to access external resource");
}
```

If the exceptions require different handling, use separate catch blocks:

```
try {
    // operation
}
catch (IOException e) {
    System.out.println("File operation failed");
}
catch (SQLException e) {
    System.out.println("Database operation failed");
}
```

11. Complete Execution Algorithm





One-line definition

Multi-catch, introduced in Java 7, allows multiple unrelated exception types to be handled by a single catch block using the | operator, reducing duplicate exception-handling code.

Java 7 — Diamond Operator (<>)

Java Version: 7

Year: 2011

Feature: Diamond Operator

UniqueClassName: DiamondOperator

1. Definition

The **Diamond Operator (<>)** was introduced in **Java 7** to reduce unnecessary repetition when creating objects of **generic classes**.

Before Java 7, programmers had to specify the generic type on **both sides** of an assignment:

```
List<String> names = new ArrayList<String>();
```

With Java 7, the type arguments on the right-hand side can be omitted:

```
List<String> names = new ArrayList<>();
```

The compiler **infers the type arguments from the context**.

Basic syntax

```
GenericType<Type> variable = new GenericType<>();
```

The <> is called the **diamond operator** because its shape resembles a diamond.

2. Why Was It Introduced?

Before Java 7

```
Map<String, Integer> map =  
    new HashMap<String, Integer>();
```

String and Integer are repeated.

Java 7+

```
Map<String, Integer> map =  
    new HashMap<>();
```

The compiler understands that:

```
HashMap<>
```

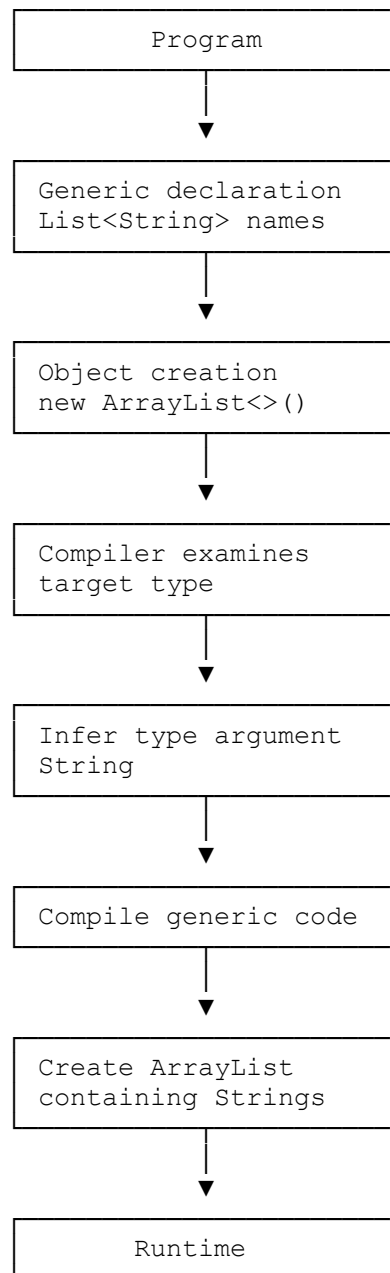
↓

String → key type

Integer → value type

because the left-hand side provides the target type.

3. Flowchart



Simplified sequence

Left-hand target type
↓
new GenericType<>()
↓
Compiler type inference

↓
Determine generic type arguments
↓
Compile
↓
Create object

4. Basic Java Code Example

```
import java.util.ArrayList;
import java.util.List;

public class DiamondOperator {

    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("Anurag");
        names.add("Rahul");
        names.add("Amit");

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Output

Anurag
Rahul
Amit

Here:

```
List<String> names = new ArrayList<>();
```

The compiler infers:

```
ArrayList<>()
  ↓
ArrayList<String>()
```

Conceptually, the programmer has written the equivalent of:

```
List<String> names = new ArrayList<String>();
```

5. Example with Map

```
import java.util.HashMap;
import java.util.Map;

public class DiamondOperator {

    public static void main(String[] args) {

        Map<Integer, String> students = new HashMap<>();

        students.put(101, "Anurag");
        students.put(102, "Rahul");
        students.put(103, "Amit");

        System.out.println(students);
    }
}
```

The compiler infers:

```
HashMap<>
↓
<Integer, String>
```

Therefore:

```
new HashMap<>();
```

is equivalent in this context to:

```
new HashMap<Integer, String>();
```

6. Before Java 7 vs Java 7

Java Version	Code	Explanation
Java 6 and earlier	List<String> list = new ArrayList<String>();	Type repeated
Java 7+	List<String> list = new ArrayList<>();	Type inferred
Left side	List<String>	Provides target type

Right side	ArrayList<>	Diamond allows omission
Benefit	Less code	Better readability

Before Java 7

```
Map<String, List<Integer>> data =
    new HashMap<String, List<Integer>>();
```

Java 7+

```
Map<String, List<Integer>> data =
    new HashMap<>();
```

The second form is particularly useful for nested generic types.

7. Generic Class Example

Consider a custom generic class:

```
class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}
```

Before Java 7:

```
Box<String> box = new Box<String>("Hello");
```

Java 7:

```
Box<String> box = new Box<>("Hello");
```

Complete program:

```
public class DiamondOperator {
    public static void main(String[] args) {
```

```

        Box<String> box = new Box<>("Hello");

        System.out.println(box.getValue());
    }
}

class Box<T> {

    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}

```

Output

Hello

8. What the Compiler Does

For:

```
List<String> names = new ArrayList<>();
```

the compiler uses the target type:

```

List<String>
  ↓
Expected generic type = String
  ↓
ArrayList<>
  ↓
Infer T = String
  ↓
ArrayList<String>

```

So the diamond does **not** mean that the object has no generic type.

It means:

The programmer does not explicitly write the type arguments because the compiler can infer them.

9. Important Limitation in Java 7

The diamond operator cannot simply be used everywhere a generic type appears.

For example, Java 7 does **not** support diamond syntax with anonymous inner classes in the way later Java versions do.

This Java 7 code is invalid:

```
Comparator<String> comparator =
    new Comparator<>() {
        public int compare(String a, String b) {
            return a.compareTo(b);
        }
    };
```

The safe Java 7 form is:

```
Comparator<String> comparator =
    new Comparator<String>() {
        public int compare(String a, String b) {
            return a.compareTo(b);
        }
    };
```

Java 9 later expanded diamond support for anonymous classes.

10. Diamond Operator and Type Safety

The diamond operator does **not remove generics** or make the collection untyped.

For example:

```
List<String> names = new ArrayList<>();
```

still means:

```
List<String>
```

↓

Only String elements

Therefore:

```
names.add("Java");    // Valid
names.add("Programming"); // Valid
```

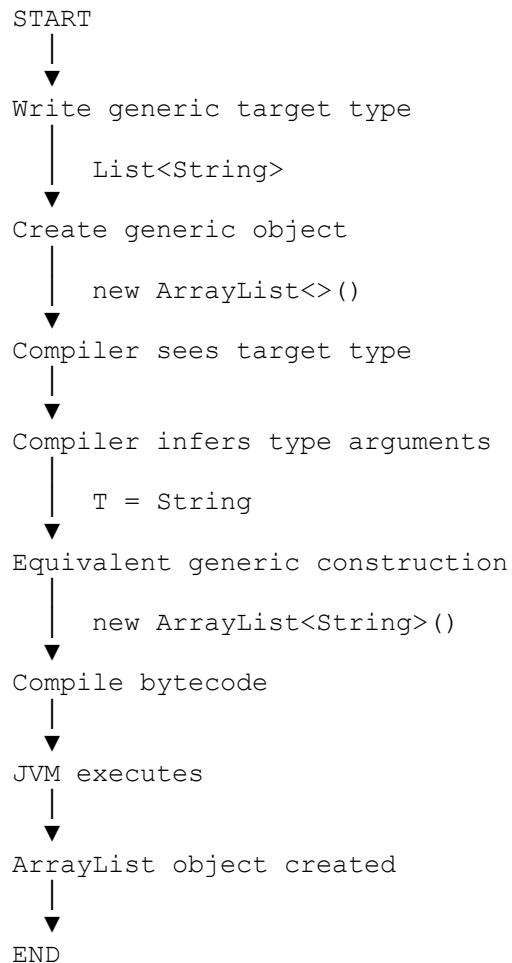
but:

```
names.add(100);
```

causes a compile-time error.

So the diamond operator is primarily a **syntax simplification**, not a change to Java's generic type-safety model.

11. Detailed Execution Sequence



12. Key Characteristics

Characteristic	Diamond Operator
Introduced	Java 7
Year	2011
Symbol	<>
Main purpose	Generic type inference
Used during	Generic object creation
Reduces	Repetition of generic type arguments
Improves	Readability and maintainability
Removes generics?	No
Removes type safety?	No
Runtime feature?	No — primarily a compile-time language feature
Works with generic classes	Yes
Works with generic constructors	Yes, subject to type-inference rules
Java 7 anonymous-class limitation	Diamond could not be used there
Later enhancement	Java 9 improved diamond support with anonymous classes

One-line definition

The Diamond Operator (<>), introduced in Java 7, allows the compiler to infer generic type arguments during object creation, reducing repetitive generic type declarations such as `new ArrayList<String>()` to `new ArrayList<>()`.

Java 7 — Strings in switch

Java Version: 7

Year: 2011

Feature: Strings in switch

UniqueClassName: StringSwitch

1. Definition

Strings in switch is a Java 7 language feature that allows a String expression to be used directly in a switch statement.

Before Java 7, Java switch supported integral types such as byte, short, char, and int (and their compatible forms), as well as enum types. A String could not be used directly.

Before Java 7

This was not allowed:

```
String day = "MONDAY";

switch (day) {
    case "MONDAY":
        System.out.println("Monday");
        break;
}
```

Java 7+

The same code is valid:

```
String day = "MONDAY";

switch (day) {
    case "MONDAY":
        System.out.println("Monday");
        break;
}
```

The feature makes code involving fixed textual choices much cleaner than a long chain of if-else if statements.

2. Basic Syntax

```
switch (stringExpression) {  
    case "value1":  
        // statements  
        break;  
  
    case "value2":  
        // statements  
        break;  
  
    default:  
        // default statements  
}
```

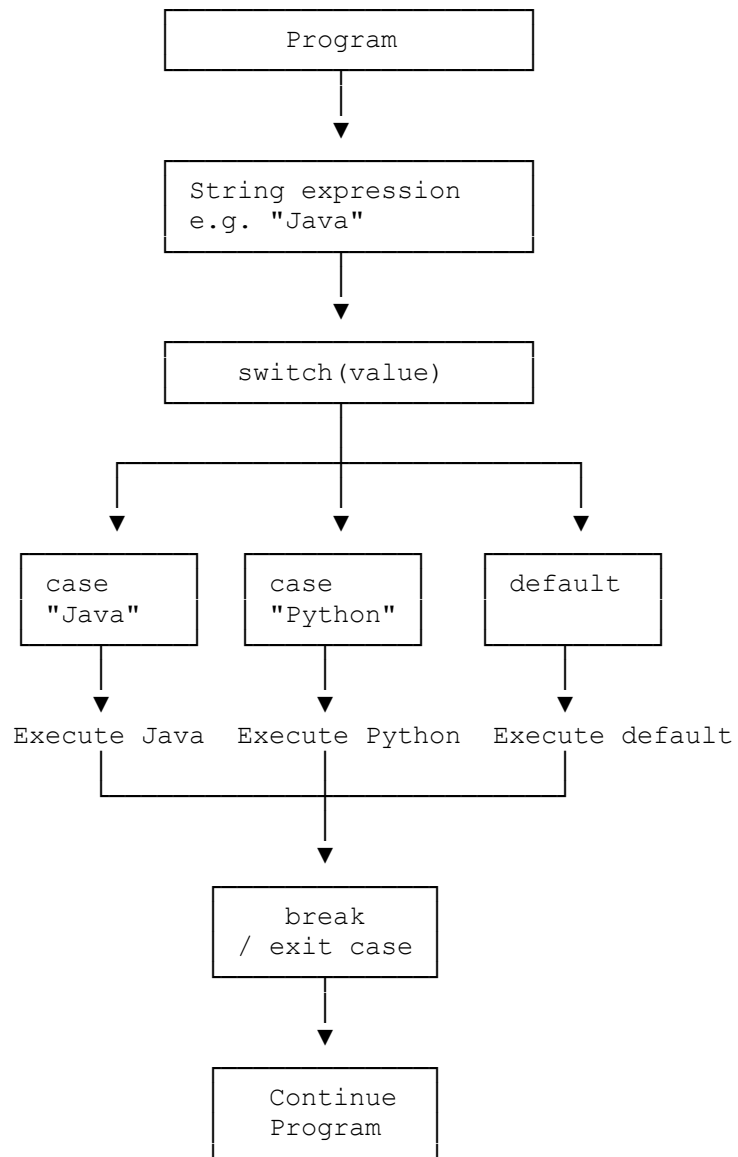
Example:

```
String language = "Java";  
  
switch (language) {  
    case "Java":  
        System.out.println("Java selected");  
        break;  
  
    case "Python":  
        System.out.println("Python selected");  
        break;  
  
    default:  
        System.out.println("Unknown language");  
}
```

Output:

Java selected

3. Flowchart



4. Java Code Example

```
public class StringSwitch {  
    public static void main(String[] args) {  
        String language = "Java";  
        switch (language) {  
            case "Java":
```

```

        System.out.println("Java programming language");
        break;

    case "Python":
        System.out.println("Python programming language");
        break;

    case "C++":
        System.out.println("C++ programming language");
        break;

    default:
        System.out.println("Unknown programming language");
    }
}
}

```

Output

Java programming language

5. Execution Flow

For:

```

String language = "Java";

switch (language) {

    case "Java":
        System.out.println("Java programming language");
        break;

    case "Python":
        System.out.println("Python programming language");
        break;

    default:
        System.out.println("Unknown programming language");
}

```

The logical sequence is:

```

language = "Java"
↓
switch(language)
↓
Compare with "Java"
↓
Match found

```

↓
Execute Java case
↓
System.out.println(...)
↓
break
↓
Exit switch
↓
Continue program

6. Multiple String Cases

Different strings can have different behavior:

```
public class StringSwitch {  
    public static void main(String[] args) {  
        String command = "start";  
        switch (command) {  
            case "start":  
                System.out.println("Application started");  
                break;  
            case "stop":  
                System.out.println("Application stopped");  
                break;  
            case "restart":  
                System.out.println("Application restarted");  
                break;  
            case "status":  
                System.out.println("Application is running");  
                break;  
            default:  
                System.out.println("Invalid command");  
        }  
    }  
}
```

Output:

Application started

7. String Matching Is Case-Sensitive

String switch uses **case-sensitive** matching.

For example:

```
String language = "java";
```

```
switch (language) {  
  
    case "Java":  
        System.out.println("Java selected");  
        break;  
  
    default:  
        System.out.println("Unknown");  
}
```

Output:

Unknown

Because:

"java" ≠ "Java"

If you want case-insensitive behavior, normalize the value first:

```
String language = "java";  
  
switch (language.toLowerCase()) {  
  
    case "java":  
        System.out.println("Java selected");  
        break;  
  
    case "python":  
        System.out.println("Python selected");  
        break;  
  
    default:  
        System.out.println("Unknown");  
}
```

8. null Consideration

A String used as the switch expression must not be null.

For example:

```
String language = null;
```

```
switch (language) {  
    case "Java":  
        System.out.println("Java");  
        break;  
}
```

This results in a `NullPointerException` at runtime.

A safe approach is to check for null first:

```
String language = null;  
  
if (language == null) {  
    System.out.println("Language is null");  
} else {  
  
    switch (language) {  
  
        case "Java":  
            System.out.println("Java");  
            break;  
  
        default:  
            System.out.println("Unknown");  
    }  
}
```

9. switch with String vs if-else

Feature	if-else	String switch
Java 7 required?	No	Yes for String switch
String comparison	Usually equals()	Built into switch
Fixed choices	Less concise	More concise
Range/complex conditions	Excellent	Not suitable
Exact string alternatives	Possible	Excellent

null handling	Can explicitly check	null causes NPE if switched directly
Readability for many fixed values	Can become lengthy	Usually cleaner

if-else version

```
if ("Java".equals(language)) {
    System.out.println("Java");
} else if ("Python".equals(language)) {
    System.out.println("Python");
} else {
    System.out.println("Unknown");
}
```

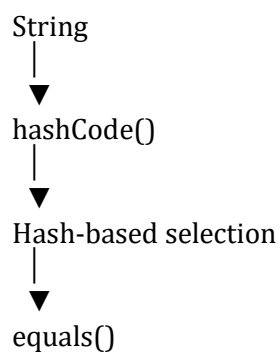
String switch version

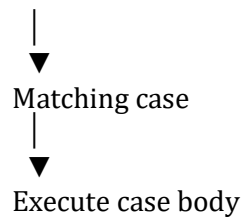
```
switch (language) {
    case "Java":
        System.out.println("Java");
        break;
    case "Python":
        System.out.println("Python");
        break;
    default:
        System.out.println("Unknown");
}
```

10. Important Internal Concept

Although the source code looks like the JVM is directly switching on a String, **Java 7 compiler implementations translate String switch into lower-level operations.**

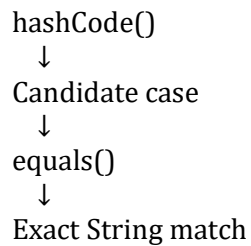
Conceptually, the compiler can use:





The important point is that **hashCode()** alone is not sufficient to determine the final match, because different strings can have the same hash code.

Therefore, equality checking is also required conceptually:

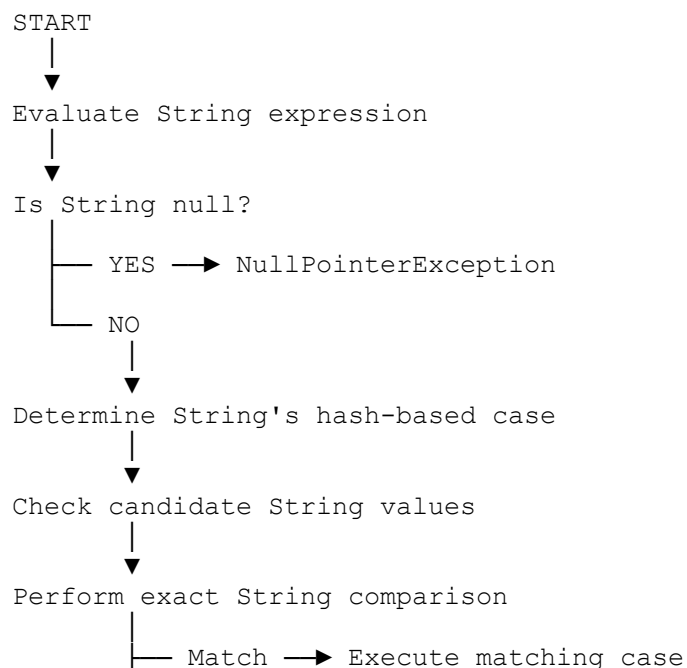


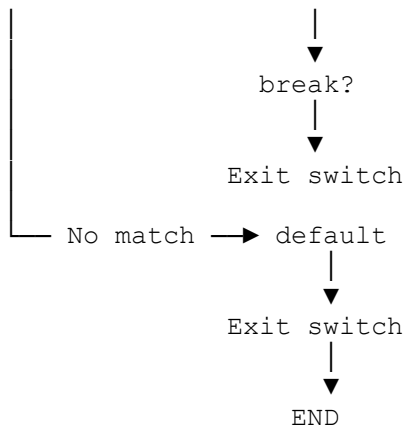
For example:

case "Java":

ultimately needs an exact equality comparison rather than relying solely on the hash value.

11. Complete Execution Algorithm





12. Java Version Comparison

Java Version	String in switch
Java 1.0	✗ Not supported
Java 1.1	✗ Not supported
Java 1.2	✗ Not supported
Java 1.3	✗ Not supported
Java 1.4	✗ Not supported
Java 5	✗ Not supported
Java 6	✗ Not supported
Java 7	✓ Introduced
Java 8	✓ Supported
Java 9	✓ Supported
Java 10+	✓ Supported
Modern Java	✓ Supported, with additional switch enhancements

13. Key Points

Point	Explanation
Feature	Strings in switch
Introduced	Java 7
Year	2011
Main purpose	Switch directly on String values
Syntax	switch (string)
Comparison	Case-sensitive
null	Direct switching on null causes NullPointerException
break	Prevents fall-through
default	Executes when no case matches
Internal concept	Hash-based dispatch plus exact equality checking
Main advantage	Cleaner alternative to long if-else chains
Runtime type	String
Compile-time feature	Yes

One-line definition

Strings in switch, introduced in Java 7, allow a String expression to be used directly in a switch statement, providing a concise way to select among multiple exact, case-sensitive string values.

Java 7 — Binary Literals

Java Version: 7

Year: 2011

Feature: Binary Literals

UniqueClassName: BinaryLiterals

1. Definition

Binary literals were introduced in **Java 7** to allow integer values to be written directly in **base-2 (binary) notation** using the prefix 0b or 0B.

Before Java 7, Java supported decimal, hexadecimal, and octal integer literals, but **binary literals were not available**.

Syntax

0b1010

or

0B1010

Here:

0b → indicates binary notation

1010 → binary digits

For example:

```
int number = 0b1010;
```

The binary value 1010_2 represents decimal 10.

2. Number-System Comparison

Number System	Base	Java Literal Example	Decimal Value
Binary	2	0b1010	10
Octal	8	012	10
Decimal	10	10	10
Hexadecimal	16	0xA	10

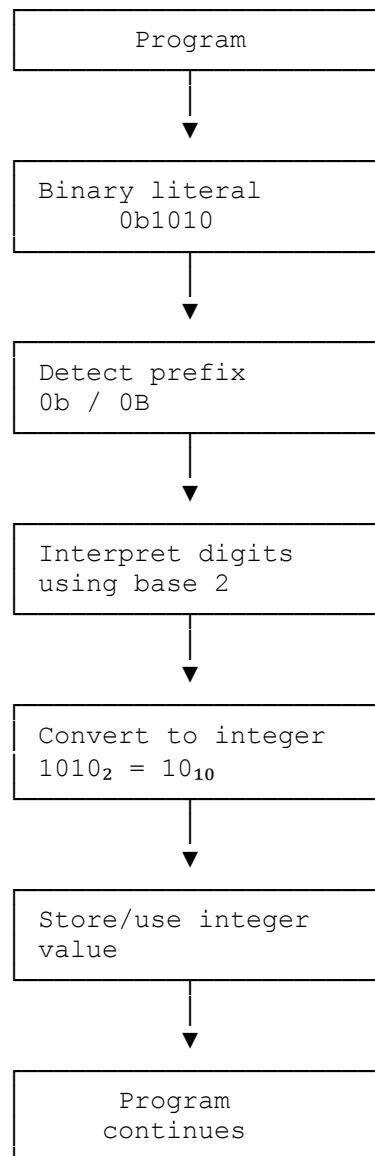
Therefore:

```
int a = 0b1010;  
int b = 012;  
int c = 10;  
int d = 0xA;
```

All four variables contain the same integer value:

10

3. Binary Literal Flowchart



Simplified sequence

0b1010
↓
Recognize 0b prefix
↓
Read binary digits
↓
Convert binary → integer value
↓
 $1010_2 = 10_{10}$
↓
Store/use value

4. Basic Java Code Example

```
public class BinaryLiterals {  
  
    public static void main(String[] args) {  
  
        int number = 0b1010;  
  
        System.out.println("Binary value: 0b1010");  
        System.out.println("Decimal value: " + number);  
    }  
}
```

Output

Binary value: 0b1010
Decimal value: 10

5. Binary-to-Decimal Calculation

Consider:

```
int number = 0b1010;
```

The binary digits represent powers of 2:

1	0	1	0	
				$0 \times 2^0 = 0$
			└──┘	$1 \times 2^1 = 2$
		└──┘	└──┘	$0 \times 2^2 = 0$
	└──┘	└──┘	└──┘	$1 \times 2^3 = 8$

Therefore:

$$8 + 0 + 2 + 0 = 10$$

So:

$$0b1010 = 10$$

6. 0b and 0B

Both lowercase and uppercase prefixes are valid.

```
int a = 0b1010;  
int b = 0B1010;
```

```
System.out.println(a);  
System.out.println(b);
```

Output:

```
10  
10
```

Therefore:

$$0b1010 == 0B1010$$

7. Binary Literals with Different Integer Types

Binary literals can be used with primitive integral types.

byte

```
byte value = 0b01111111;  
  
System.out.println(value);
```

Output:

```
127
```

short

```
short value = 0b0000000011111111;
```

```
System.out.println(value);
```

Output:

511

int

```
int value = 0b10101010;
```

```
System.out.println(value);
```

Output:

170

long

For a long literal, use the L suffix when necessary:

```
long value = 0b10101010L;
```

```
System.out.println(value);
```

Output:

170

8. Binary Literals with Underscores

Java 7 also introduced **underscores in numeric literals**, making long binary values easier to read.

For example:

```
int number = 0b1010_1010_1010_1010;
```

Instead of:

```
int number = 0b1010101010101010;
```

The underscores have no mathematical effect.

```
0b1010_1010_1010
```

↓

```
0b101010101010
```

Both represent the same value.

Example

```
public class BinaryLiterals {  
    public static void main(String[] args) {  
        int number = 0b1010_1010;  
        System.out.println(number);  
    }  
}
```

Output:

170

9. Practical Bitwise Example

Binary literals are particularly useful when working with **bitwise operations**.

```
public class BinaryLiterals {  
    public static void main(String[] args) {  
        int a = 0b1100;  
        int b = 0b1010;  
  
        int andResult = a & b;  
        int orResult  = a | b;  
        int xorResult = a ^ b;  
  
        System.out.println("AND = " + andResult);  
        System.out.println("OR  = " + orResult);  
        System.out.println("XOR = " + xorResult);  
    }  
}
```

Calculation

```
    1100  
AND 1010  
----  
    1000
```

Therefore:

$1000_2 = 8$

Similarly:

```
    1100
OR  1010
----
    1110
11102 = 14
```

And:

```
    1100
XOR 1010
----
    0110
01102 = 6
```

Output

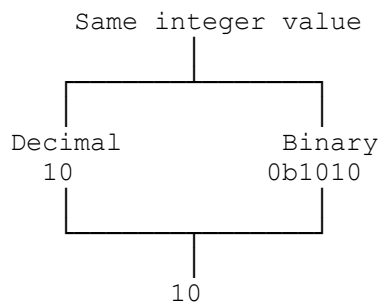
```
AND = 8
OR  = 14
XOR = 6
```

10. Binary Literal vs Decimal Literal

These two declarations produce exactly the same value:

```
int decimal = 10;
int binary  = 0b1010;
```

Conceptually:



The difference is only in **how the programmer represents the number in source code**.

11. Important Restrictions

Binary literals can contain only binary digits:

0
1

Valid:

```
int a = 0b101010;
```

Invalid:

```
int b = 0b102010;
```

because 2 is not a binary digit.

Also, the 0b/0B prefix is required:

```
int a = 0b1010; // Correct
```

Without the prefix:

```
int b = 1010; // Decimal 1010, NOT binary
```

12. Binary Literals and Java Version

Java Version	Binary Literals
Java 1.0	✗ Not supported
Java 1.1	✗
Java 1.2	✗
Java 1.3	✗
Java 1.4	✗
Java 5	✗
Java 6	✗
Java 7	✓ Introduced
Java 8	✓

Java 9	✓
Java 10+	✓
Modern Java	✓

13. Binary Literal + Underscore Example

Java 7's two related numeric-literal enhancements can be used together:

```
public class BinaryLiterals {
    public static void main(String[] args) {
        int permissions = 0b110_101_101;

        System.out.println(
            "Decimal value = " + permissions
        );
    }
}
```

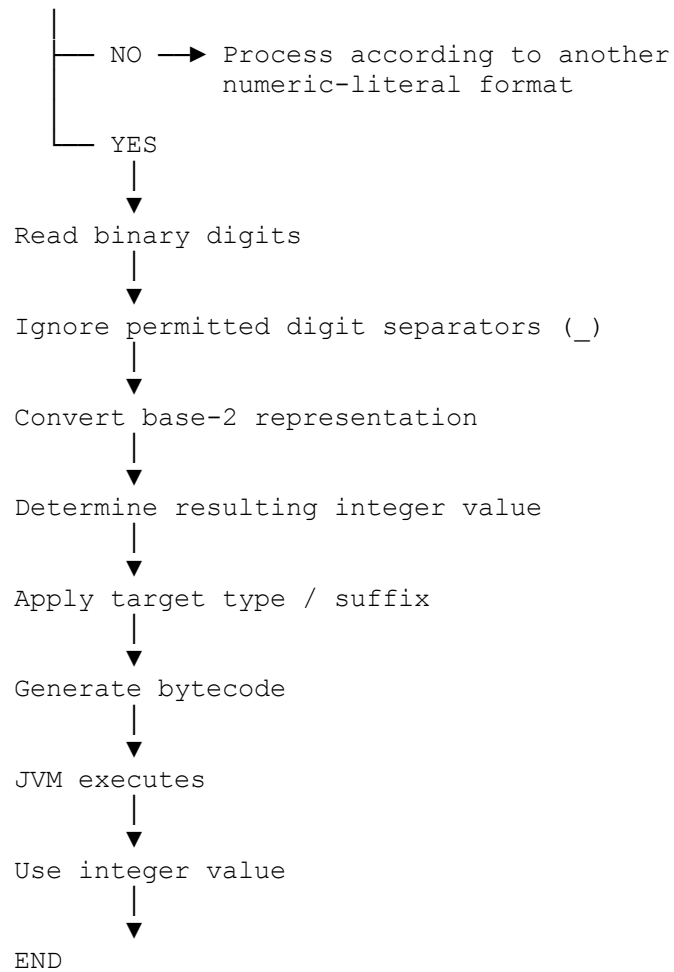
The compiler interprets:

```
0b110_101_101
    ↓
0b110101101
    ↓
429
```

The underscore exists purely for **source-code readability**.

14. Complete Execution Algorithm

```
START
  ↓
Read source code
  ↓
Find numeric literal
  ↓
Does it start with 0b or 0B?
```



15. Key Points

Point	Explanation
Feature	Binary Literals
Introduced	Java 7
Year	2011
Prefix	0b or 0B
Number system	Base 2
Valid digits	0 and 1
Example	0b1010

Decimal equivalent	10
Can use _?	Yes, since Java 7
Main purpose	Make binary values directly expressible in source code
Particularly useful for	Bit manipulation, flags, masks, low-level programming
Runtime representation	Normal primitive integer value
Compile-time feature	Yes

One-line definition

Binary literals, introduced in Java 7, allow integer values to be written directly in binary notation using the 0b or 0B prefix, making bit patterns and bitwise operations easier to read and understand.

Java 7 — Underscores in Numeric Literals

Java Version: 7

Year: 2011

Feature: Underscores in Numeric Literals

UniqueClassName: UnderscoreNumbers

1. Definition

Underscores in numeric literals were introduced in **Java 7** to make long numeric values easier to read.

Java allows underscores (_) to be placed **between digits** of numeric literals.

For example:

```
int number = 1_000_000;
```

is exactly equivalent to:

```
int number = 1000000;
```

The underscores are **ignored by the compiler** and do not change the numeric value.

Basic concept

1_000_000
↓
1000000
↓
Integer value = 1,000,000

2. Why Were Underscores Introduced?

Large numbers can be difficult to read:

long population = 1234567890L;

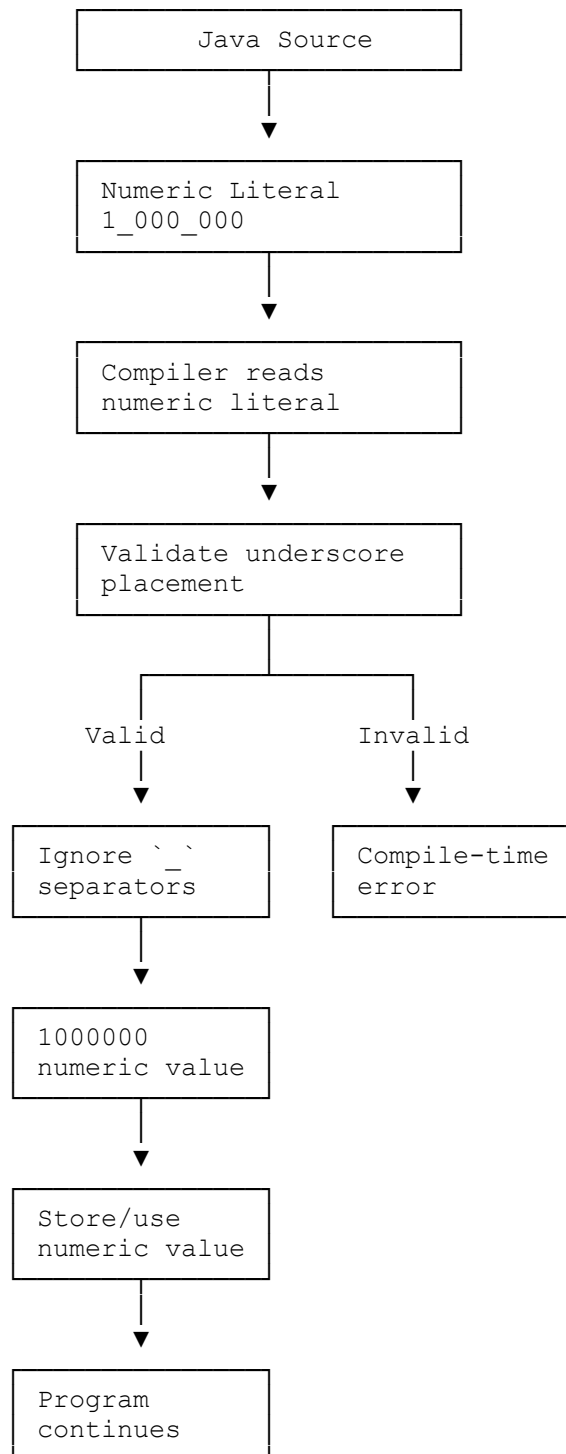
With underscores:

long population = 1_234_567_890L;

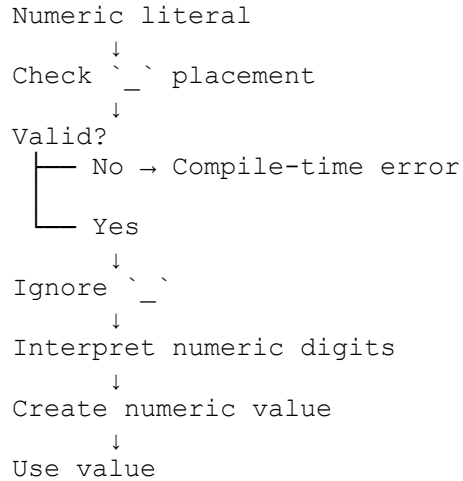
The second form makes the digit groups much easier to recognize.

Without underscores	With underscores
1000000	1_000_000
1000000000	1_000_000_000
1234567890L	1_234_567_890L
0xFF00FF00	0xFF00_FF00
0b1010101010101010	0b1010_1010_1010_1010

3. Flowchart



Simplified sequence



4. Basic Java Code Example

```
public class UnderscoreNumbers {  
  
    public static void main(String[] args) {  
  
        int number = 1_000_000;  
  
        System.out.println("Number = " + number);  
    }  
}
```

Output

Number = 1000000

Notice that Java does **not** print the underscores.

The underscore exists only in the source code.

5. Different Numeric Types

Underscores can be used with various integer and floating-point literals.

int

```
int population = 1_000_000;
```

long

```
long distance = 9_876_543_210L;
```

float

```
float price = 1_234.56f;
```

double

```
double amount = 1_234_567.89;
```

short

```
short value = 32_000;
```

byte

```
byte value = 1_00;
```

The underscores do not change the underlying numeric values.

6. Complete Java Example

```
public class UnderscoreNumbers {  
    public static void main(String[] args) {  
        int population = 1_000_000;  
        long distance = 9_876_543_210L;  
        float price = 1_234.56f;  
        double amount = 12_345_678.90;  
  
        System.out.println("Population = " + population);  
        System.out.println("Distance   = " + distance);  
        System.out.println("Price      = " + price);  
        System.out.println("Amount    = " + amount);  
    }  
}
```

Output

```
Population    = 1000000  
Distance      = 9876543210  
Price         = 1234.56
```

Amount = 1.23456789E7

The exact formatting of floating-point output can vary depending on how it is printed, but the numeric value is unchanged.

7. Underscores with Binary Literals

Java 7 also introduced **binary literals**.

Underscores can make binary values much easier to understand:

```
int value = 0b1010_1010;
```

Without separators:

```
int value = 0b10101010;
```

Both represent:

$$10101010_2 = 170_{10}$$

Example

```
public class UnderscoreNumbers {  
    public static void main(String[] args) {  
        int value = 0b1010_1010;  
        System.out.println(value);  
    }  
}
```

Output:

170

8. Underscores with Hexadecimal Literals

Underscores can also improve the readability of hexadecimal numbers.

```
int color = 0xFF00_FF00;
```

This is equivalent to:

```
int color = 0xFF00FF00;
```

The underscore simply separates logical groups.

9. Underscores with Octal Literals

They can also be used with octal numbers:

```
int value = 0123_456;
```

The underscore does not change the octal value.

10. Underscores with Floating-Point Values

Underscores can be placed between digits in floating-point literals.

```
double value = 123_456.789_012;
```

Equivalent to:

```
double value = 123456.789012;
```

Example:

```
public class UnderscoreNumbers {  
    public static void main(String[] args) {  
        double value = 123_456.789_012;  
        System.out.println(value);  
    }  
}
```

Output:

```
123456.789012
```

11. Where Underscores Are Allowed

The general rule is:

An underscore can appear between digits, but it cannot be placed arbitrarily inside a numeric literal.

Valid examples:

```
int a = 1_000;  
int b = 10_00_000;  
int c = 0b1010_1010;  
int d = 0xFF_FF;  
double e = 12_345.67_89;
```

12. Where Underscores Are Not Allowed

An underscore cannot be placed directly next to certain parts of a numeric literal.

Not directly after the base prefix

Invalid:

```
int value = 0b_1010;
```

Valid:

```
int value = 0b1010;
```

Not directly before a suffix

Invalid:

```
long value = 1_000_L;
```

Valid:

```
long value = 1_000L;
```

Not directly after the decimal point

Invalid:

```
double value = 123._456;
```

Valid:

```
double value = 123.456;
```

Not directly before the decimal point

Invalid:

```
double value = 123_.456;
```

Not at the beginning of the digits

Invalid:

```
int value = _1000;
```

This is not interpreted as a numeric literal with an underscore.

13. Valid vs Invalid Examples

Literal	Valid?	Reason
1_000	✓	Between digits
10_00_000	✓	Between digits
0b1010_1010	✓	Between binary digits
0xFF_FF	✓	Between hexadecimal digits
123_456.789	✓	Between digits
123.456_789	✓	Between digits
0b_1010	✗	Immediately after 0b
0x_FF	✗	Immediately after 0x
1_000_L	✗	Immediately before suffix
123_.456	✗	Immediately before decimal point
123._456	✗	Immediately after decimal point

14. Underscores Do Not Affect the Value

Consider:

```
int a = 1000000;
```

```
int b = 1_000_000;
```

Then:

```
System.out.println(a == b);
```

Output:

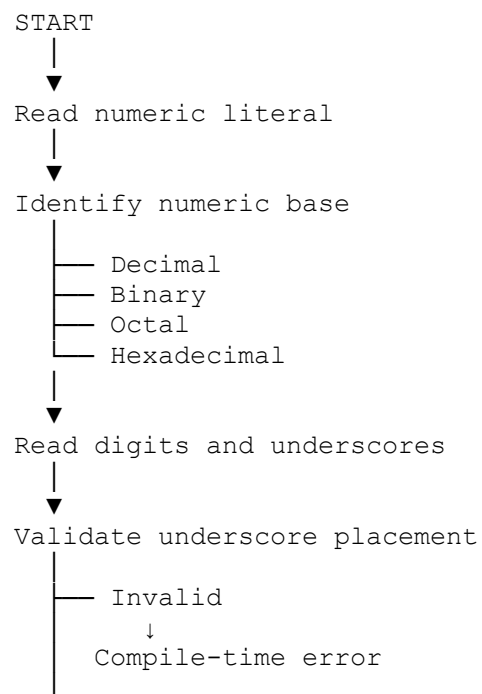
true

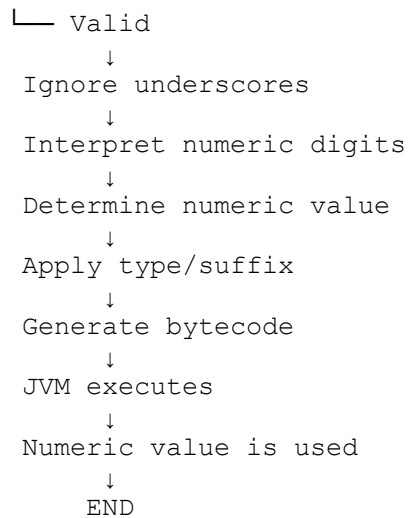
Conceptually:

```
1_000_000
  ↓
underscore separators removed
  ↓
1000000
  ↓
numeric value
  ↓
1,000,000
```

The underscore is therefore a **source-code readability feature**, not part of the runtime numeric value.

15. Complete Execution Algorithm





16. Java Version Timeline

Java Version	Underscores in Numeric Literals
Java 1.0	✗ Not supported
Java 1.1	✗
Java 1.2	✗
Java 1.3	✗
Java 1.4	✗
Java 5	✗
Java 6	✗
Java 7	✓ Introduced
Java 8	✓
Java 9	✓
Java 10+	✓
Modern Java	✓

17. Key Points

Point	Explanation
Feature	Underscores in Numeric Literals
Introduced	Java 7
Year	2011
Symbol	-
Purpose	Improve readability of long numeric literals
Runtime effect	None
Compiler behavior	Ignores valid underscore separators
Decimal numbers	✓ Supported
Binary numbers	✓ Supported
Octal numbers	✓ Supported
Hexadecimal numbers	✓ Supported
Floating-point numbers	✓ Supported
Type safety	Unchanged
Invalid placement	Causes compile-time error

One-line definition

Underscores in numeric literals, introduced in Java 7, allow underscores to be placed between digits of numeric literals to improve readability without changing the actual numeric value.

Java 7 — NIO.2

Java Version: 7

Year: 2011

Feature: NIO.2

UniqueClassName: NIO2

1. Definition

NIO.2 is the major enhancement to Java's I/O and file-system APIs introduced in **Java 7** through **JSR 203**. It introduced the `java.nio.file` API, providing a more powerful and flexible way to work with files, directories, file attributes, symbolic links, file trees, and file-system change notifications. ([Oracle](#))

The most important NIO.2 classes and interfaces include:

- `Path` — represents a file-system path.
- `Paths` — creates `Path` objects.
- `Files` — provides static methods for file and directory operations.
- `FileVisitor` / `SimpleFileVisitor` — supports file-tree traversal.
- `WatchService` — monitors directories for file-system changes.
- `DirectoryStream` — iterates through directory entries.
- `FileSystem` / `FileSystems` — represents and accesses file systems.
- `java.nio.file.attribute` — provides file metadata and attribute views. ([Oracle Docs](#))

A useful way to remember NIO.2 is:

```
NIO.2
├── Path
├── Files
├── DirectoryStream
├── FileVisitor
├── WatchService
├── FileSystem
└── File Attributes
```

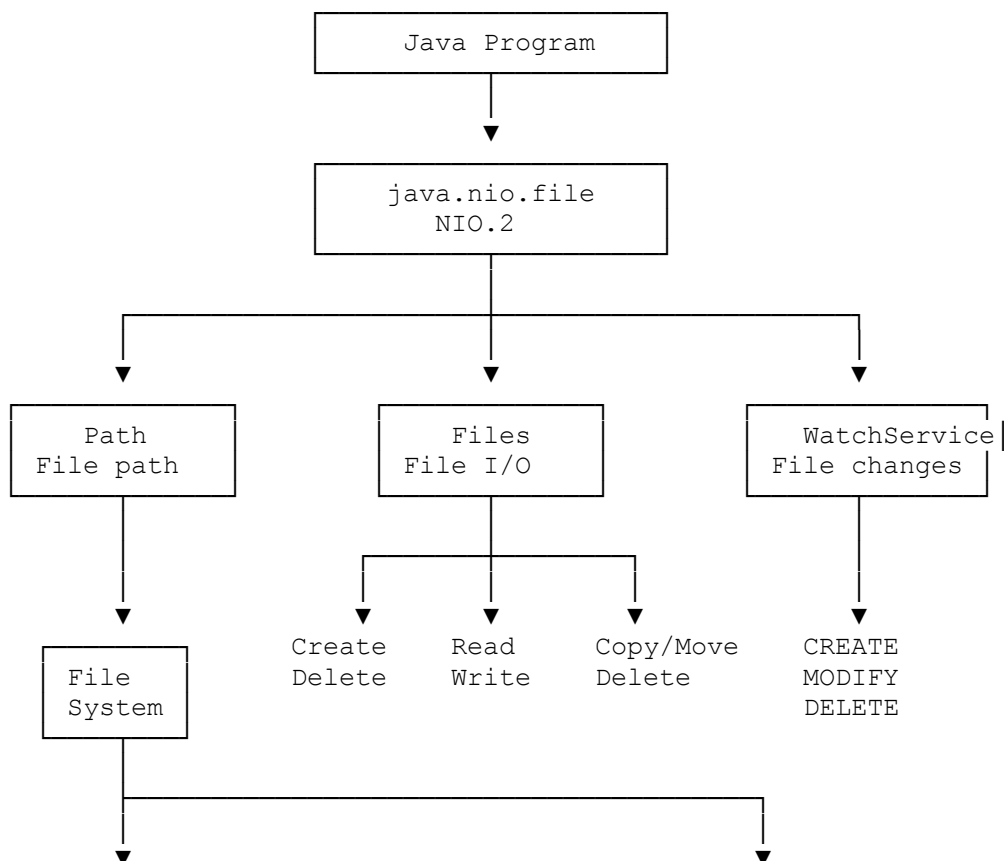
2. NIO vs NIO.2

Feature	NIO (Java 1.4)	NIO.2 (Java 7)
Introduced	Java 1.4	Java 7
Main focus	Buffers, channels, non-blocking I/O	Modern file-system API

Main file abstraction	File was still commonly used	Path
File operations	More limited	Files utility API
File tree traversal	Manual/recursive logic	walkFileTree()
File change notification	Limited	WatchService
Symbolic links	Limited	Explicit support
File attributes	Limited	Rich attribute API
File-system providers	Limited	Pluggable providers
Main package	java.nio	java.nio.file

NIO itself dates to Java 1.4; **NIO.2 is the Java 7 enhancement**, particularly the file-system API in java.nio.file. ([Oracle](#))

3. NIO.2 Architecture / Flowchart



File Tree FileVisitor walkFileTree
--

File Attributes Permissions Size/Time/etc.
--

4. Path — The Central NIO.2 Abstraction

Path represents the location of a file or directory in a file system. It is broadly the NIO.2 counterpart to the older `java.io.File` abstraction. ([Oracle](#))

Example:

```
Path path = Paths.get("data.txt");
```

Conceptually:

```
String path
↓
Paths.get(...)
↓
Path object
↓
File-system operation
```

A Path does **not necessarily mean that the file exists**. It represents a location/reference to a file-system object. ([Oracle](#))

5. Basic Java Code Example — Path and Files

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class NIO2 {

    public static void main(String[] args) {

        Path path = Paths.get("data.txt");

        try {
            Files.write(
                path,
                "Hello from Java NIO.2".getBytes("UTF-8")
            );

            String content = new String(
```

```

        Files.readAllBytes(path),
        "UTF-8"
    );

    System.out.println("File: " + path);
    System.out.println("Content: " + content);

} catch (IOException e) {
    e.printStackTrace();
}
}
}

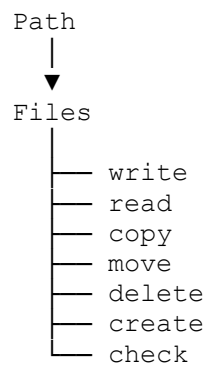
```

Output

File: data.txt

Content: Hello from Java NIO.2

The important relationship is:



Files is a utility class containing static methods for operating on files, directories, and other file-system objects. ([Oracle Docs](#))

6. File Operations with Files

```

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class NIO2 {

    public static void main(String[] args) {

        Path source = Paths.get("source.txt");
        Path target = Paths.get("target.txt");
    }
}

```

```

    try {
        Files.write(
            source,
            "NIO.2 Example".getBytes("UTF-8")
        );

        Files.copy(source, target);

        System.out.println(
            "Target exists: " + Files.exists(target)
        );

        Files.delete(target);

        System.out.println(
            "Target exists after delete: "
            + Files.exists(target)
        );

    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

Conceptually:

```

Create source.txt
↓
Write data
↓
Copy to target.txt
↓
Check target
↓
Delete target

```

7. Directory Operations

NIO.2 also provides convenient directory operations.

```

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class NIO2 {

```

```

public static void main(String[] args) {

    Path directory = Paths.get("documents");

    try {

        if (!Files.exists(directory)) {
            Files.createDirectory(directory);
        }

        System.out.println(
            "Directory exists: "
            + Files.exists(directory)
        );

    } catch (IOException e) {
        e.printStackTrace();
    }

}

```

8. DirectoryStream

NIO.2 provides `DirectoryStream` for iterating over directory entries. It is designed to avoid some of the scalability problems of returning a complete array of directory names/files.

```

import java.io.IOException;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class NIO2 {

    public static void main(String[] args) {

        Path directory = Paths.get(".");

        try (DirectoryStream<Path> stream =
            Files.newDirectoryStream(directory)) {

            for (Path entry : stream) {
                System.out.println(entry);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }

    }

}

```

Flow:

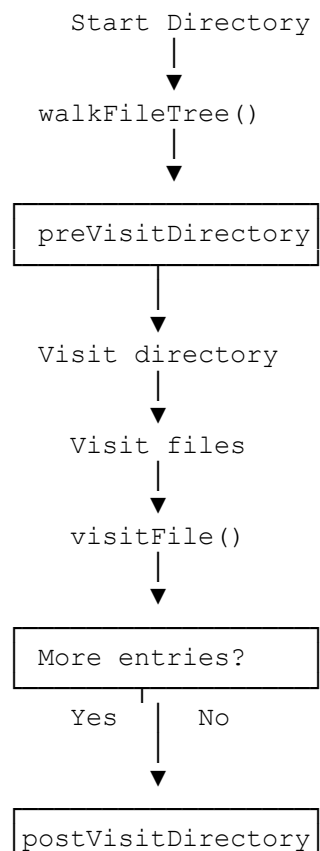
```
Directory
↓
Files.newDirectoryStream()
↓
DirectoryStream<Path>
↓
Iterate entries
↓
Process each Path
↓
Close stream
```

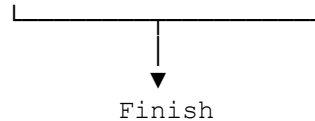
9. File-Tree Traversal — walkFileTree()

One of the important NIO.2 features is recursive file-tree traversal.

`Files.walkFileTree()` works with a `FileVisitor` and invokes visitor methods as it walks through files and directories. ([Oracle Docs](#))

Flowchart





Java example

```
import java.io.IOException;
import java.nio.file.FileVisitResult;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.SimpleFileVisitor;
import java.nio.file.attribute.BasicFileAttributes;

public class NIO2 {

    public static void main(String[] args) {

        Path start = Paths.get(".");

        try {

            Files.walkFileTree(
                start,
                new SimpleFileVisitor<Path>() {

                    @Override
                    public FileVisitResult visitFile(
                        Path file,
                        BasicFileAttributes attrs)
                        throws IOException {

                        System.out.println(file);

                        return FileVisitResult.CONTINUE;
                    }
                }
            );

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

This can be used for recursive operations such as copying, moving, deleting, or processing an entire directory tree. ([Oracle Docs](#))

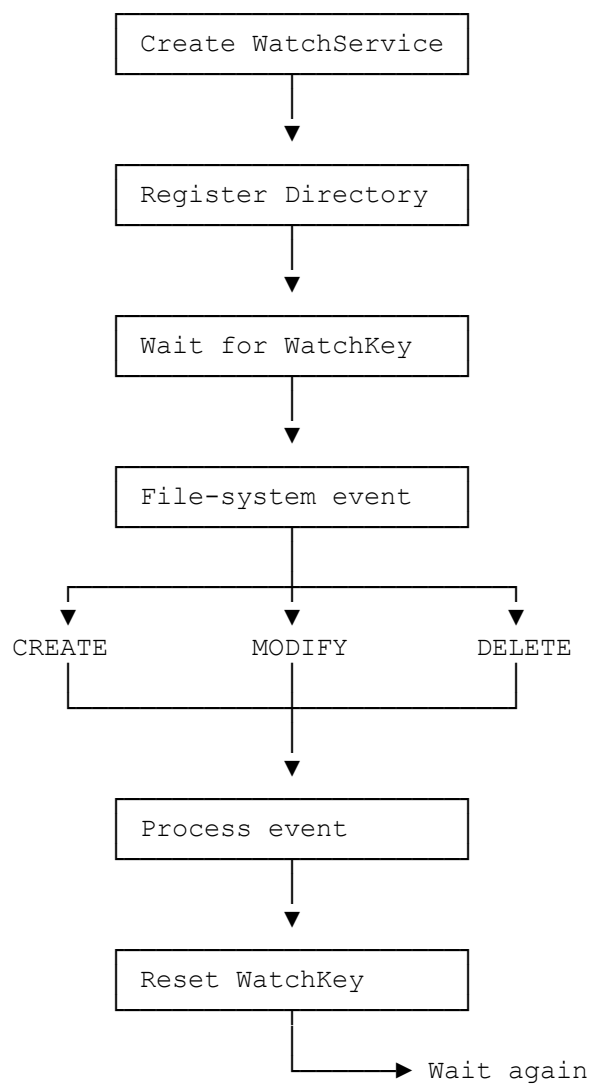
10. WatchService — Monitor File-System Changes

NIO.2 introduced WatchService for receiving notifications when registered directories change. Standard events include:

- ENTRY_CREATE
- ENTRY_DELETE
- ENTRY_MODIFY

([Oracle Docs](#))

Flowchart



Java Code

```
import java.io.IOException;
import java.nio.file.*;

public class NIO2 {

    public static void main(String[] args) {

        Path directory = Paths.get(".");

        try (WatchService watcher =
            FileSystems.getDefault().newWatchService()) {

            directory.register(
                watcher,
                StandardWatchEventKinds.ENTRY_CREATE,
                StandardWatchEventKinds.ENTRY_MODIFY,
                StandardWatchEventKinds.ENTRY_DELETE
            );

            System.out.println(
                "Watching: " + directory.toAbsolutePath()
            );

            WatchKey key = watcher.take();

            for (WatchEvent<?> event : key.pollEvents()) {

                System.out.println(
                    "Event: " + event.kind()
                );

                System.out.println(
                    "File: " + event.context()
                );
            }

            key.reset();

        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

The general WatchService workflow is to create a watcher, register directories and event types, wait for a WatchKey, process pending events, reset the key, and continue waiting. ([Oracle](#))

11. File Attributes

NIO.2 provides APIs for accessing file metadata such as:

- File size
- Creation time
- Last-modified time
- File type
- Permissions
- Other file-system-specific attributes

([Oracle Docs](#))

Example:

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class NIO2 {

    public static void main(String[] args) {

        Path path = Paths.get("data.txt");

        try {

            System.out.println(
                "Exists: " + Files.exists(path)
            );

            System.out.println(
                "Regular file: "
                + Files.isRegularFile(path)
            );

            System.out.println(
                "Directory: "
                + Files.isDirectory(path)
            );

            System.out.println(
                "Size: "
                + Files.size(path)
                + " bytes"
            );

            System.out.println(
                "Last modified: "
                + Files.getLastModifiedTime(path)
            );

        } catch (IOException e) {
```

```

        e.printStackTrace();
    }
}

```

12. java.io.File vs NIO.2 Path

Old API	NIO.2
File	Path
File.exists()	Files.exists(path)
File.isDirectory()	Files.isDirectory(path)
File.delete()	Files.delete(path)
File.renameTo()	Files.move()
File.listFiles()	DirectoryStream / other NIO.2 APIs
Manual recursion	Files.walkFileTree()
Limited metadata	Rich attribute APIs
Limited file-change notification	WatchService

NIO.2 also provides interoperability between the old File API and the new Path API through `File.toPath()`. ([Oracle Docs](#))

Example:

```

import java.io.File;
import java.nio.file.Path;

public class NIO2 {

    public static void main(String[] args) {

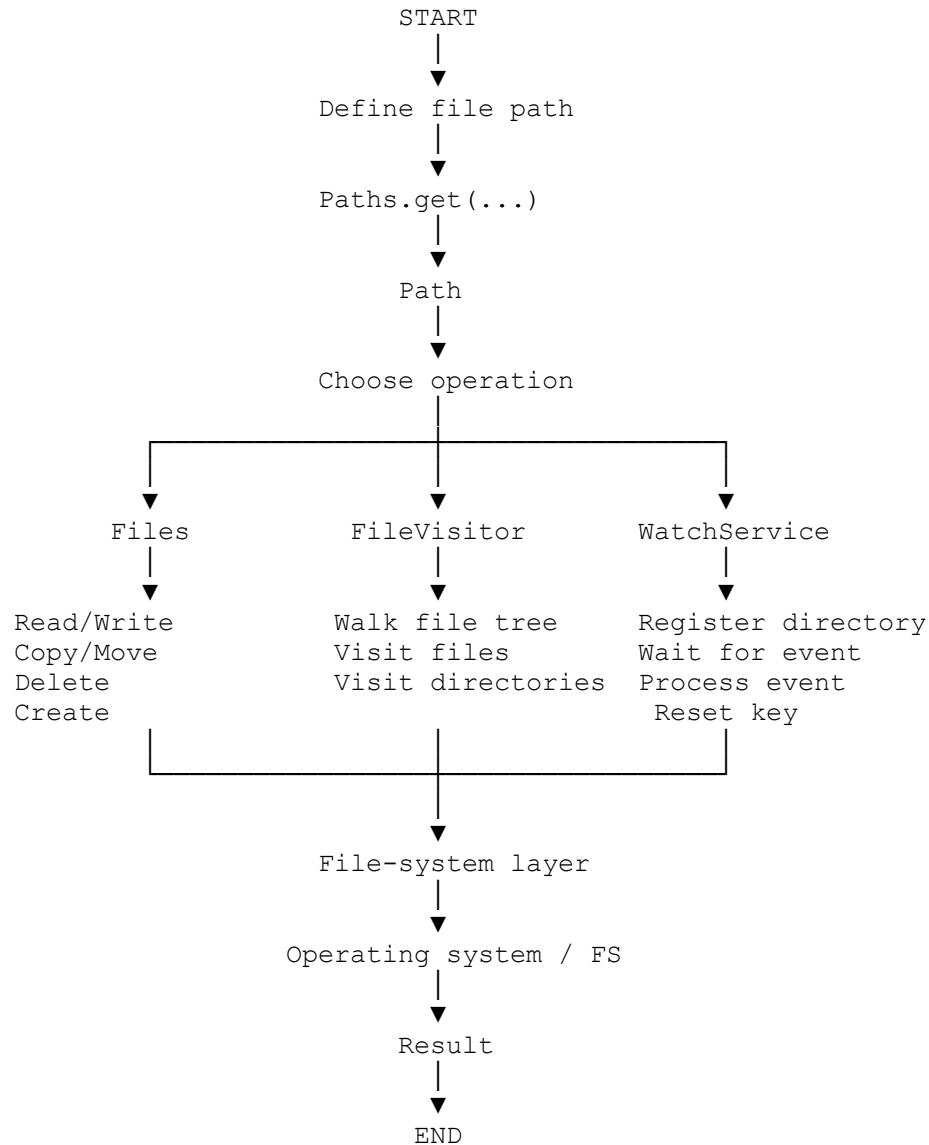
        File file = new File("data.txt");

        Path path = file.toPath();

        System.out.println(path);
    }
}

```

13. Complete NIO.2 Execution Model



14. Main NIO.2 Components

Component	Purpose
Path	Represents a file-system path
Paths	Creates Path objects
Files	Performs common file/directory operations
DirectoryStream	Iterates directory entries
FileVisitor	Defines file-tree traversal behavior
SimpleFileVisitor	Convenient default FileVisitor implementation
FileVisitResult	Controls file-tree traversal
WatchService	Monitors registered directories
WatchKey	Represents a registration with a watch service
WatchEvent	Represents a file-system event
FileSystem	Represents a file system
FileSystems	Provides file-system access/factory methods
BasicFileAttributes	Provides basic file metadata
LinkOption	Controls symbolic-link handling
StandardCopyOption	Controls copy/move behavior
StandardOpenOption	Controls file opening/creation
java.nio.file.attribute	File metadata/attribute APIs
java.nio.file.spi	File-system provider SPI

These components are part of the Java 7 `java.nio.file` API and its related packages. ([Oracle Docs](#))

15. Important NIO.2 Features Introduced in Java 7

Feature	Purpose
Path	Modern representation of file-system paths
Files	Convenient file operations
File-tree walking	Recursive directory processing
FileVisitor	Custom file-tree operations
WatchService	File/directory change notifications
Symbolic-link support	Work with symbolic links
File attributes	Access file metadata and permissions
DirectoryStream	Efficient directory iteration
File-system providers	Support pluggable/custom file systems
Improved exceptions	More specific file-system failure information

Oracle describes JSR 203/NIO.2 as addressing long-standing file-I/O limitations, including copying/moving, symbolic links, permissions, file attributes, directory handling, and file-change notification. ([Oracle](#))

16. Key Points

Point	Explanation
Feature	NIO.2
Introduced	Java 7
Year	2011
Specification	JSR 203
Main package	java.nio.file
Central abstraction	Path
Main utility class	Files

File-tree API	FileVisitor / walkFileTree()
Change monitoring	WatchService
Directory iteration	DirectoryStream
Metadata	java.nio.file.attribute
Symbolic links	Supported
Custom file systems	Supported through provider SPI
Relationship to NIO	NIO.2 is the major Java 7 enhancement to Java's NIO/file-system APIs

One-line definition

NIO.2, introduced in Java 7 through JSR 203, is the modern Java file-system and I/O API centered around Path, Files, file-tree traversal, file attributes, symbolic links, and WatchService for monitoring file-system changes. ([Oracle](#))

Java 7 — Fork/Join Framework

Java Version: 7

Year: 2011

Feature: Fork/Join Framework

UniqueClassName: ForkJoinFramework

1. Definition

The **Fork/Join Framework** was introduced in **Java 7** to support **parallel processing of computationally intensive tasks**.

It is designed around the **divide-and-conquer** technique:

```
Large Task
↓
Divide into smaller tasks
↓
Fork smaller tasks
↓
Execute tasks in parallel
↓
Join results
↓
Combine results
↓
Final Result
```

The framework is primarily located in:

`java.util.concurrent`

Important classes include:

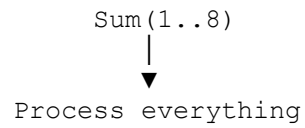
Class / Interface	Purpose
ForkJoinPool	Manages worker threads and executes fork/join tasks
ForkJoinTask<V>	Base class for fork/join tasks
RecursiveTask<V>	Task that returns a result
RecursiveAction	Task that does not return a result
ForkJoinWorkerThread	Worker thread used by the pool

2. Core Idea: Divide and Conquer

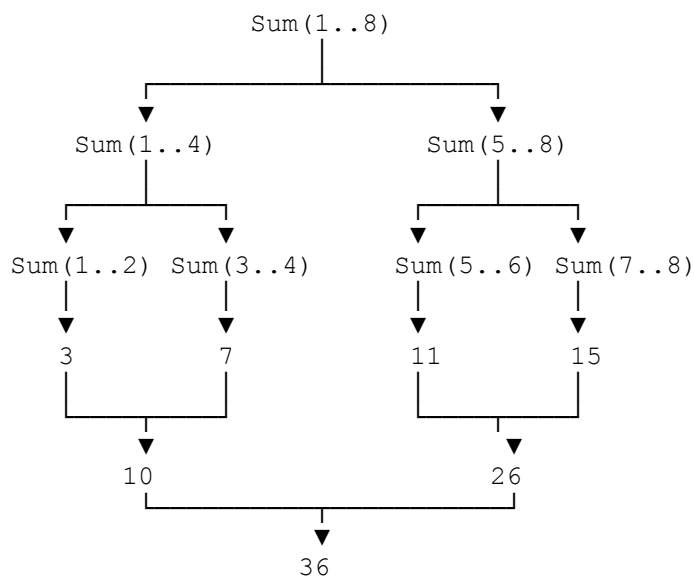
Suppose we need to calculate the sum of:

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$$

Instead of processing everything as one task:

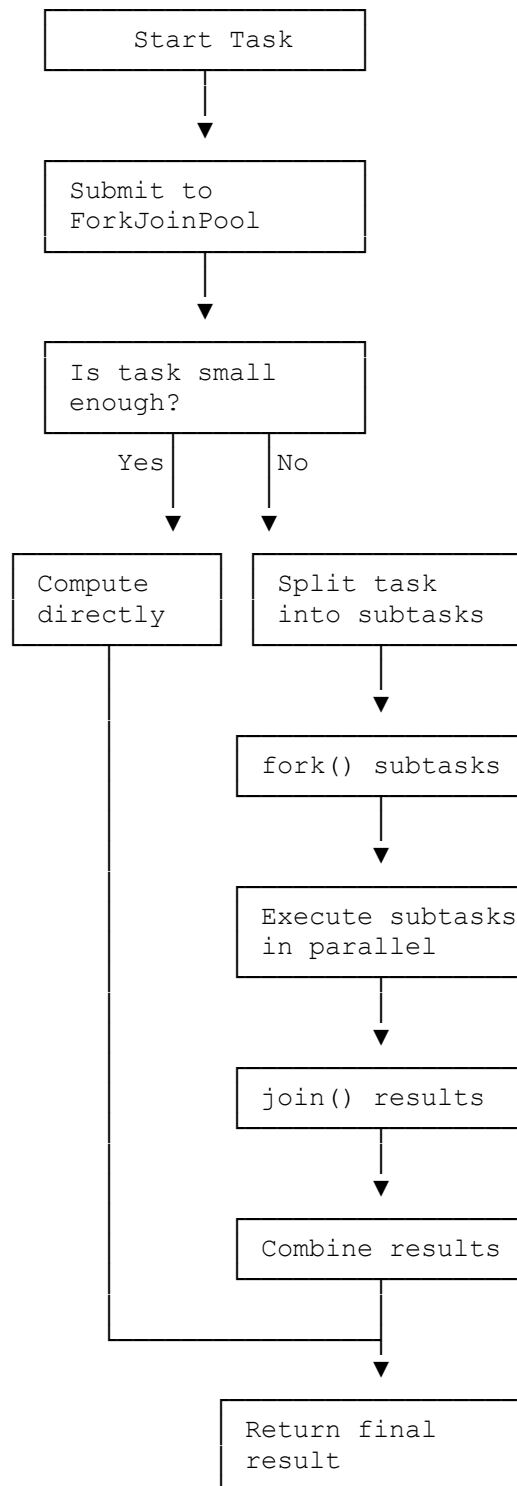


Fork/Join can divide it:



The smaller tasks can be executed concurrently when the system has suitable worker capacity.

3. Fork/Join Flowchart



4. The Three Most Important Operations

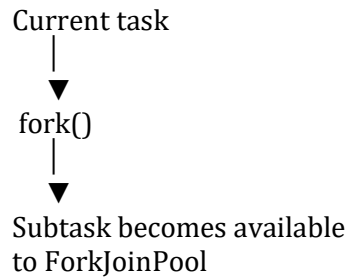
Fork/Join revolves around three concepts:

fork()

Schedules a task for asynchronous execution.

```
task.fork();
```

Conceptually:

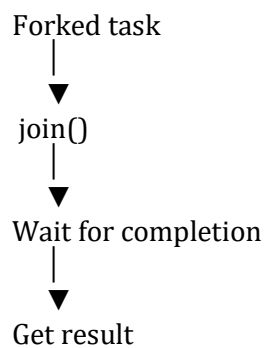


join()

Waits for a forked task to complete and obtains its result.

```
int result = task.join();
```

Conceptually:



compute()

Contains the actual task logic.

```
protected Integer compute() {  
    // computation  
}
```

The fundamental pattern is:

```
compute()
├── Small enough?
│   └── Compute directly
└── Too large?
    ├── Split
    ├── fork()
    ├── fork()
    ├── join()
    └── combine
```

5. RecursiveTask vs RecursiveAction

There are two commonly used task types.

Type	Returns result?	Use
RecursiveTask<V>	Yes	Calculations producing a result
RecursiveAction	No	Operations performed for their side effects

For example:

`RecursiveTask<Integer>`

can calculate a sum and return an Integer.

Whereas:

`RecursiveAction`

can perform an operation such as modifying an array without returning a value.

6. Java Code Example — RecursiveTask

Let's calculate the sum of an integer array using Fork/Join.

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

public class ForkJoinFramework {
```

```

static class SumTask extends RecursiveTask<Integer> {

    private static final int THRESHOLD = 2;

    private final int[] numbers;
    private final int start;
    private final int end;

    SumTask(int[] numbers, int start, int end) {
        this.numbers = numbers;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Integer compute() {

        int length = end - start;

        // Base case
        if (length <= THRESHOLD) {

            int sum = 0;

            for (int i = start; i < end; i++) {
                sum += numbers[i];
            }

            return sum;
        }

        // Divide
        int middle = start + length / 2;

        SumTask leftTask =
            new SumTask(numbers, start, middle);

        SumTask rightTask =
            new SumTask(numbers, middle, end);

        // Fork
        leftTask.fork();
        rightTask.fork();

        // Join
        int leftResult = leftTask.join();
        int rightResult = rightTask.join();

        // Combine
        return leftResult + rightResult;
    }
}

public static void main(String[] args) {

    int[] numbers = {

```

```

        1, 2, 3, 4,
        5, 6, 7, 8
    };

    ForkJoinPool pool = new ForkJoinPool();

    SumTask task =
        new SumTask(numbers, 0, numbers.length);

    int result = pool.invoke(task);

    System.out.println("Sum = " + result);

    pool.shutdown();
}
}

```

Output

Sum = 36

7. How the Example Executes

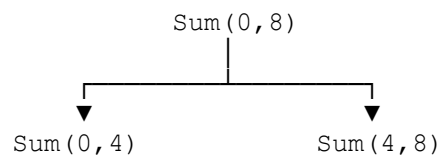
The original array is:

1 2 3 4 5 6 7 8

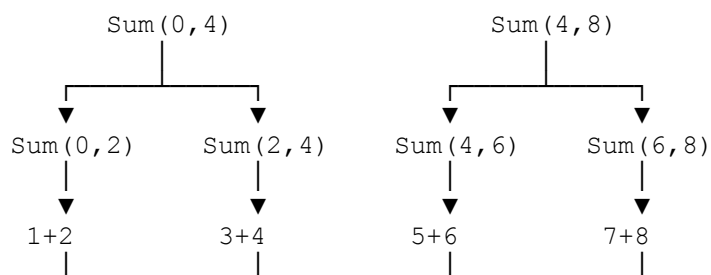
The task initially represents:

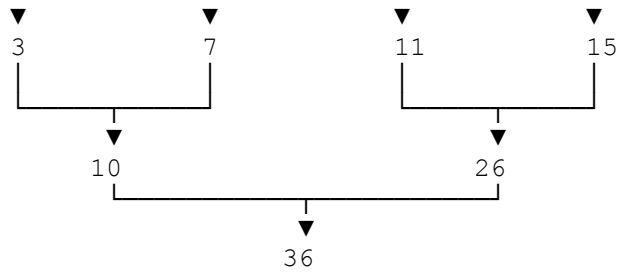
Sum(0, 8)

Since the task contains more than THRESHOLD elements, it splits:



Then:





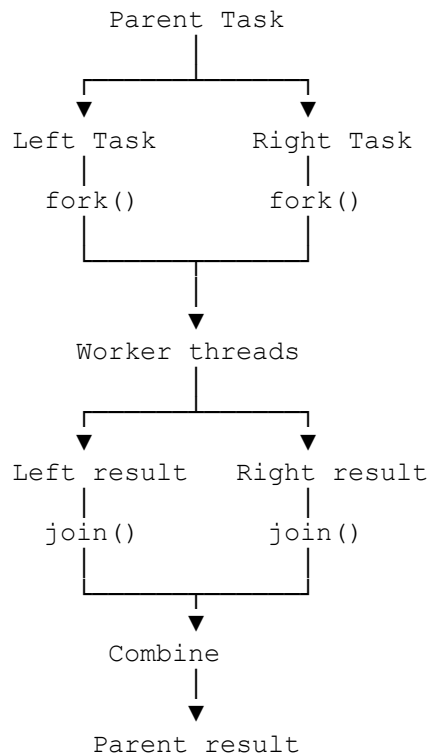
8. fork() and join() Sequence

For the example:

```
leftTask.fork();
rightTask.fork();

int leftResult = leftTask.join();
int rightResult = rightTask.join();
```

the conceptual sequence is:



fork() does not mean "create a new OS thread for every task." The ForkJoinPool manages a bounded set of worker threads and schedules tasks onto them.

9. ForkJoinPool

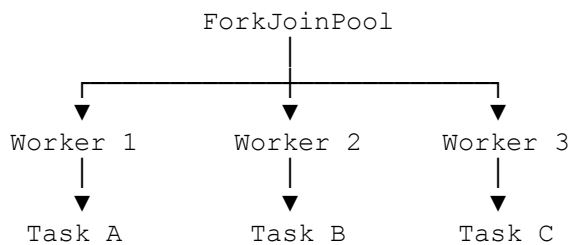
ForkJoinPool is the execution engine for Fork/Join tasks.

Example:

```
ForkJoinPool pool = new ForkJoinPool();
```

```
int result = pool.invoke(task);
```

Conceptually:



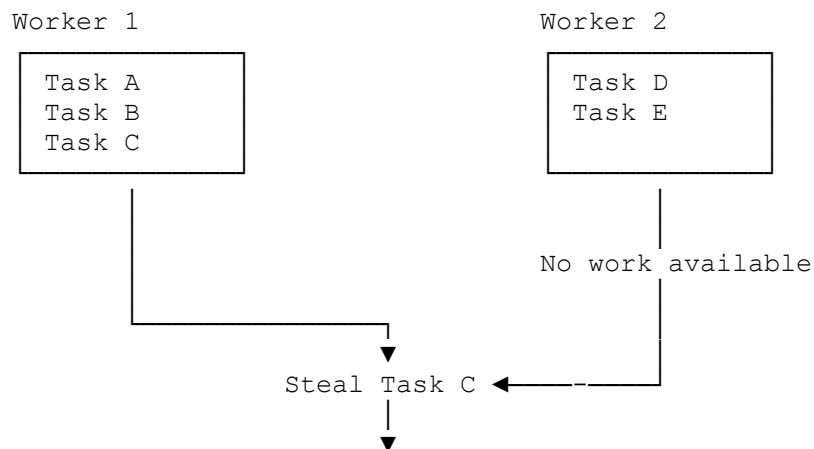
The pool manages worker threads and task scheduling.

10. Work-Stealing

One of the key design concepts behind ForkJoinPool is **work-stealing**.

A worker generally has its own deque of tasks. When a worker runs out of work, it can steal tasks from another worker's deque.

Conceptually:



Worker 2
executes C

This helps keep worker threads busy when workloads are uneven.

11. RecursiveAction Example

Not every parallel task needs to return a value.

For example, suppose we want to print or process elements in parallel:

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

public class ForkJoinFramework {

    static class PrintTask extends RecursiveAction {

        private static final int THRESHOLD = 2;

        private final int[] numbers;
        private final int start;
        private final int end;

        PrintTask(int[] numbers, int start, int end) {
            this.numbers = numbers;
            this.start = start;
            this.end = end;
        }

        @Override
        protected void compute() {

            int length = end - start;

            if (length <= THRESHOLD) {

                for (int i = start; i < end; i++) {
                    System.out.println(numbers[i]);
                }

                return;
            }

            int middle = start + length / 2;

            PrintTask left =
                new PrintTask(numbers, start, middle);

            PrintTask right =
                new PrintTask(numbers, middle, end);
```

```

        invokeAll(left, right);
    }
}

public static void main(String[] args) {

    int[] numbers = {
        1, 2, 3, 4,
        5, 6, 7, 8
    };

    ForkJoinPool pool = new ForkJoinPool();

    PrintTask task =
        new PrintTask(numbers, 0, numbers.length);

    pool.invoke(task);

    pool.shutdown();
}
}

```

Here:

RecursiveAction

is appropriate because the task doesn't produce a result.

12. invokeAll() Shortcut

Instead of manually doing:

```

leftTask.fork();
rightTask.fork();

```

```

int leftResult = leftTask.join();
int rightResult = rightTask.join();

```

tasks can often be submitted together:

```

invokeAll(leftTask, rightTask);

```

For RecursiveTask, you can then obtain their results:

```

invokeAll(leftTask, rightTask);

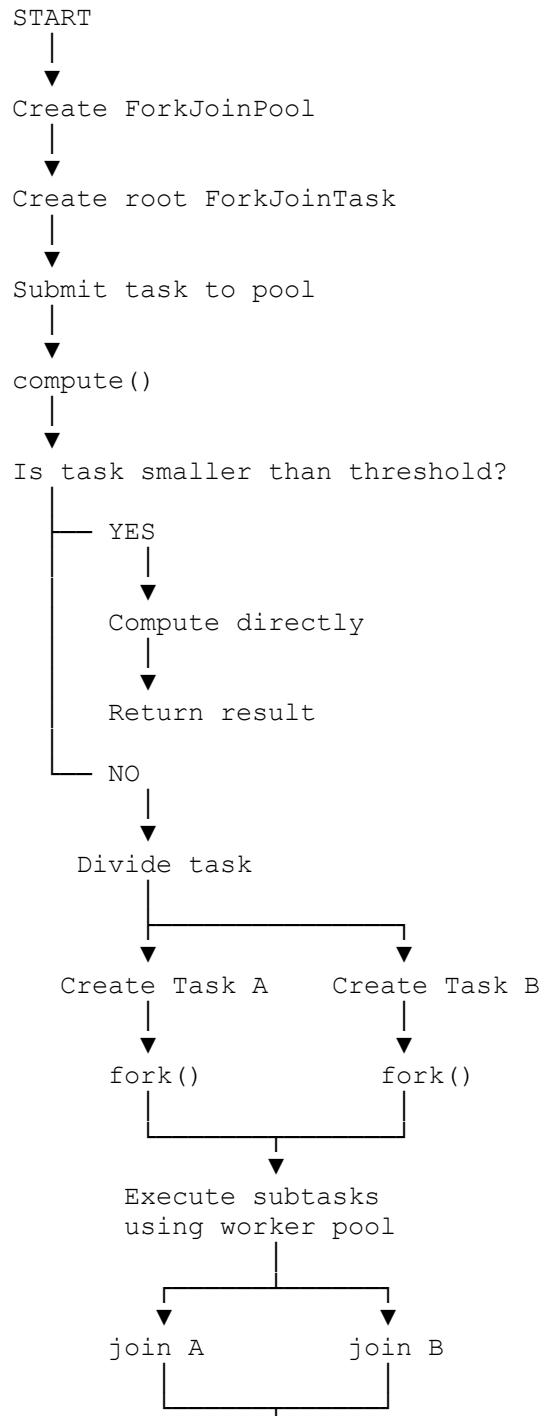
int leftResult = leftTask.join();

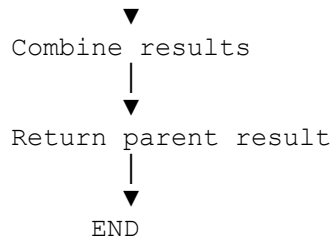
```

```
int rightResult = rightTask.join();
```

This makes the divide-and-conquer structure easier to express.

13. Complete Fork/Join Algorithm





14. When Should Fork/Join Be Used?

Fork/Join is particularly suitable when:

- A large computational problem can be recursively divided.
- Subtasks are relatively independent.
- Subtasks can execute concurrently.
- The workload can benefit from multiple CPU cores.
- The task has a meaningful threshold at which sequential processing becomes cheaper.

Typical examples include:

Large array
↓
Split array
↓
Process portions concurrently

Examples of algorithms that can potentially benefit:

- Parallel array processing
- Recursive numerical calculations
- Parallel sorting
- Searching large data structures
- Image/data processing
- Divide-and-conquer algorithms

It is **not automatically faster** than sequential execution. Creating and scheduling many tiny tasks introduces overhead, which is why choosing an appropriate **threshold** is important.

15. Fork/Join vs Traditional Threading

Feature	Traditional Threads	Fork/Join
Introduced	Earlier Java versions	Java 7
Main abstraction	Thread	ForkJoinTask

Thread management	Programmer-oriented	Pool-managed
Divide-and-conquer	Manual	Built in conceptually
Worker pool	ExecutorService commonly used	ForkJoinPool
Work stealing	No standard Thread mechanism	Yes, ForkJoinPool
Recursive tasks	More cumbersome	Natural fit
Result tasks	Manual coordination	RecursiveTask<V>
No-result tasks	Runnable	RecursiveAction
Best suited for	General concurrent work	CPU-intensive recursive parallelism

16. Important Distinction

Fork/Join does **not** mean:

One task

↓

Create thousands of threads

Instead:

One ForkJoinPool

↓

Limited worker threads

↓

Many lightweight tasks

↓

Workers execute tasks

↓

Work stealing balances workload

This is one of the most important concepts to understand.

17. Key Points

Point	Explanation
Feature	Fork/Join Framework
Introduced	Java 7
Year	2011
Package	java.util.concurrent
Main pool	ForkJoinPool
Base task	ForkJoinTask
Result-producing task	RecursiveTask<V>
No-result task	RecursiveAction
Main strategy	Divide and conquer
fork()	Schedules a task for asynchronous execution
join()	Waits for/completes a task and obtains its result
compute()	Contains task computation
invokeAll()	Helps execute multiple subtasks
Scheduling technique	Work stealing
Best suited for	CPU-intensive parallel computations
Important tuning concept	Task threshold/granularity

One-line definition

The Fork/Join Framework, introduced in Java 7, is a `java.util.concurrent` framework for executing recursively split, CPU-intensive tasks in parallel using `ForkJoinPool`, `RecursiveTask`, `RecursiveAction`, `fork()`, `join()`, and work-stealing.